

NY Vehicle Collision Analysis

CS210 Final Project

Arushi Singhal (as4966) | Venkat Aditya Mullagiri (vm662) | Yug Patel (yp517)

Problem Statement

The goal of this project was to predict the accident severity based on various time-related factors. We analyzed a sample of 100k (out of more than 2 million) NYC motor vehicle collisions to identify any patterns related to the crashes. Using this, we assessed the accident severity to understand the impacts for further analysis.

The Data

NYC OpenData

Menu

AboutDataRelated Content

ActionsExport

Motor Vehicle Collisions - CrashesPublic Safety

The Motor Vehicle Collisions crash table contains details on the crash event. Each row represents a crash event. The Motor Vehicle Collisions data tables contain information from all police reported motor vehicle collisions in NYC. The police report (MV104-AN) is...

Read more

Last Updated
May 5, 2026

Data Provided By
Police Department (NYPD)

About this Dataset

Updated
May 5, 2026

Data Last Updated
May 5, 2026

Metadata Last Updated
April 19, 2021

Date Created
April 28, 2014

Views
855K

Downloads
369K

Data Collection

Data Collection

Motor Vehicle Collisions

Dataset Information

Agency

Police Department (NYPD)

Legislative Compliance

Update

Update Frequency

Daily

Automation

Yes

Date Made Public

5/7/2014

- Available to public
- Verified source - NYPD
- Dataset is updated daily
- More than 2.2M rows of data

Data Selection

1. 2.2M rows in full dataset — not practical to use all
2. API hard cap: 50,000 rows per request
3. Solution: 2 API calls using offset → 100k rows total
4. Ordered by crash date DESC → most recent data first

```
# BATCH 1: fetch first 50k rows
▼ params1 = {
    "$limit": 50000,
    "$offset": 0,
    "$order": "crash_date DESC"
}
response1 = requests.get(url, params=params1, timeout=60)
print("Batch 1 status code:", response1.status_code)

# parse the response into a dataframe
batch1 = pd.read_csv(io.StringIO(response1.text), low_memory=False)
print("Batch 1 rows:", len(batch1))

# BATCH 2: fetch next 50k rows using offset
▼ params2 = {
    "$limit": 50000,
    "$offset": 50000,
    "$order": "crash_date DESC"
}
response2 = requests.get(url, params=params2, timeout=60)
print("Batch 2 status code:", response2.status_code)

# parse the second batch
batch2 = pd.read_csv(io.StringIO(response2.text), low_memory=False)
print("Batch 2 rows:", len(batch2))
```

Sampling & Saving

1. Merged both batches → 100,000 rows
2. Shuffled with random seed 42 → reproducibility
3. Saved as Collisions_sample_100k.csv
4. Data provenance documented

```
Batch 1 status code: 200
Batch 1 rows: 50000
Batch 2 status code: 200
Batch 2 rows: 50000
Combined total: 100000

   crash_date crash_time ... vehicle_type_code_4 vehicle_type_code_5
0  2026-05-03T00:00:00.000  15:00 ...           NaN           NaN
1  2026-05-03T00:00:00.000  14:00 ...           NaN           NaN
2  2026-05-03T00:00:00.000  14:35 ...           NaN           NaN
3  2026-05-03T00:00:00.000  18:34 ...           NaN           NaN
4  2026-05-03T00:00:00.000   6:00 ...           NaN           NaN

[5 rows x 29 columns]
['crash_date', 'crash_time', 'borough', 'zip_code', 'latitude', 'longitude', 'location', 'on_street_name', 'off_street_name', 'cross_street_name', 'number_of_persons_injured', 'number_of_persons_killed', 'number_of_pedestrians_injured', 'number_of_pedestrians_killed', 'number_of_cyclist_injured', 'number_of_cyclist_killed', 'number_of_motorist_injured', 'number_of_motorist_killed', 'contributing_factor_vehicle_1', 'contributing_factor_vehicle_2', 'contributing_factor_vehicle_3', 'contributing_factor_vehicle_4', 'contributing_factor_vehicle_5', 'collision_id', 'vehicle_type_code1', 'vehicle_type_code2', 'vehicle_type_code_3', 'vehicle_type_code_4', 'vehicle_type_code_5']
{'source': 'https://data.cityofnewyork.us/resource/h9gi-nx95.csv', 'collected_on': Timestamp('2026-05-06 23:07:28.191885'), 'collection_method': '2x GET requests using requests library (50k rows each)', 'total_rows': 100000, 'transformations': 'Combined 2 batches, shuffled rows'}

saved 100,000 rows -> data\Collisions_sample_100k.csv
```

Data Cleaning

1. Raw dataset: 100,000 rows, 29 columns
2. Free-text fields, inconsistent formatting, invalid values
3. Goal: clean, standardize, and prepare for analysis

Raw data — Collisions_sample_100k.csv before cleaning

3 rows · 29 columns · messy date format · NaN values scattered throughout

#	crash_date	crash_time	borough	zip_code	latitude	longitude
0	2025-06-12T00:00:00.000	9:49	BROOKLYN	11226.0	40.652863	-73.9582
1	2025-05-26T00:00:00.000	13:50	QUEENS	11368.0	0.0	0.0
2	2026-02-03T00:00:00.000	14:04	BROOKLYN	11225.0	40.662014	-73.961426

continued ↓

#	location	on_street_name	off_street_name	cross_street_name
0	\n, \n(40.652863, -73.9582)	NaN	NaN	2164 CATON AVE
1	\n, \n(0.0, 0.0)	NaN	NaN	32-09 HARPER ST
2	\n, \n(40.662014, -73.961426)	NaN	NaN	505 FLATBUSH AVE

continued ↓

#	no_persons_injur...	no_persons_killed	no_ped_injured	no_ped_killed	no_cyclist_inj...	no_cyclist_kil...	no_motorist_inju...
0	1	0.0	0	0	1	0	0
1	0	0.0	0	0	0	0	0
2	0	0.0	0	0	0	0	0

continued ↓

#	contributing_factor_vehicle_1	contributing_factor_vehicle_2	contributing_f...	contributing_f...	contributing_f...
0	Driver Inattention/Distractio	Driver Inattention/Distractio	NaN	NaN	NaN
1	Driver Inattention/Distractio	Unspecified	NaN	NaN	NaN
2	Driver Inexperience	Unspecified	NaN	NaN	NaN

continued ↓

#	collision_id	vehicle_type_code1	vehicle_type_...	vehicle_type...	vehicle_type...	vehicle_type...
0	4819870	Station Wagon/Sport Utility Vehicle	Bike	NaN	NaN	NaN
1	4816011	Sedan	Sedan	NaN	NaN	NaN
2	4876072	Station Wagon/Sport Utility Vehicle	Box Truck	NaN	NaN	NaN

Standardizing & Selecting Columns

1. Renamed all columns: caps + spaces
→ lowercase + underscores
2. Kept 19 of 29 columns — only what's relevant
3. Dropped: street names, zip code, collision ID, location string

```
# renaming columns
df.columns = df.columns.str.strip().str.lower().str.replace(" ", "_")
print(f"columns found: {list(df.columns)}")
```


● kept ● dropped ● newly engineered

Before — raw CSV

29 columns

- crash_date
- crash_time
- borough
- zip_code
- latitude
- longitude
- location
- on_street_name
- off_street_name
- cross_street_name
- number_of_persons_injured
- number_of_persons_killed
- number_of_pedestrians_injured
- number_of_pedestrians_killed
- number_of_cyclist_injured
- number_of_cyclist_killed
- number_of_motorist_injured
- number_of_motorist_killed
- contributing_factor_vehicle_1
- contributing_factor_vehicle_2
- contributing_factor_vehicle_3
- contributing_factor_vehicle_4
- contributing_factor_vehicle_5
- collision_id
- vehicle_type_code1
- vehicle_type_code2
- vehicle_type_code3
- vehicle_type_code4
- vehicle_type_code5

After — cleaned CSV

32 columns

- crash_date
- crash_time
- borough
- latitude
- longitude
- number_of_persons_injured
- number_of_persons_killed
- number_of_pedestrians_injured
- number_of_pedestrians_killed
- number_of_cyclist_injured
- number_of_cyclist_killed
- number_of_motorist_injured
- number_of_motorist_killed
- contributing_factor_vehicle_1
- contributing_factor_vehicle_2
- vehicle_type_code1
- vehicle_type_code2
- vehicle_type_code3
- vehicle_type_code4
- vehicle_type_code5
- hour
- month
- day_of_week
- is_rush_hour
- is_weekend
- is_night
- factor_group
- num_vehicles
- total_injuries
- total_fatalities
- severity_score
- is_severe

-9

columns dropped

+12

features engineered

29 → 32

total columns

Cleaning Messy Values - Regex

1. Crash time: only keep valid HH:MM format
2. Borough: strip symbols, validate against 5 NYC boroughs → else 'Unknown'
3. Vehicle type: 50+ variations → 8 clean categories
4. Coordinates: null out anything outside NYC's geographic range

```
valid_boroughs = {"BRONX", "BROOKLYN", "MANHATTAN", "QUEENS", "STATEN ISLAND"}  
def clean_borough(val):  
    val = str(val).strip().upper()  
    val = re.sub(r"[^A-Z\s]", "", val)  
    val = re.sub(r"\s+", " ", val)  
    return val if val in valid_boroughs else "Unknown"
```

```
patterns = [  
    (r"\bSEDAN\b|\b4 DOOR\b|\b2 DOOR\b", "SEDAN"),  
    (r"\bTAXI\b|\bCAB\b|\bYELLOW CAB\b", "TAXI"),  
    (r"\bSUV\b|\bSPORT UTILITY\b", "SUV"),  
    (r"\bBUS\b|\bSCHOOL BUS\b|\bMTA\b", "BUS"),  
    (r"\bTRUCK\b|\bPICKUP\b|\bPICK UP\b", "TRUCK"),  
    (r"\bMOTORCYCLE\b|\bSCOOTER\b|\bMOTORBIKE\b", "MOTORCYCLE"),  
    (r"\bBICYCLE\b|\bBIKE\b|\bEBIKE\b|\bE BIKE\b", "BICYCLE"),  
    (r"\bVAN\b|\bMINIVAN\b|\bMINI VAN\b", "VAN"),  
]
```

Feature Engineering

—

Engineering Time Features

1. Extracted: hour, month, day of week from date/time
2. is_rush_hour $\rightarrow$ 1 if 6–10am or 4–8pm
3. is_weekend $\rightarrow$ 1 if Saturday or Sunday
4. is_night $\rightarrow$ 1 if midnight to 5am

```
df["hour"]      = pd.to_datetime(df["crash_time"].astype(str), errors="coerce").dt.hour
df["month"]     = df["crash_date"].dt.month
df["day_of_week"] = df["crash_date"].dt.day_name()
```

```
df["is_rush_hour"] = df["hour"].apply(
    lambda h: 1 if pd.notna(h) and (6 <= h < 10 or 16 <= h < 20) else 0
)
df["is_weekend"] = df["day_of_week"].apply(
    lambda d: 1 if d in ("Saturday", "Sunday") else 0
)
df["is_night"] = df["hour"].apply(
    lambda h: 1 if pd.notna(h) and (0 <= h < 5) else 0
)
```

Factor Grouping & Vehicle Count

1. 50+ unique contributing factor values → 6 clean groups
2. Distraction, Speeding, Alcohol/Drugs, Failure to Yield, Weather/Road, Fatigue
3. num_vehicles: count of non-null vehicle columns per row

```
def group_factor(val):  
    val = str(val).upper()  
    if re.search(r"INATTENTION|DISTRACT|PHONE|TEXTING", val):  
        return "Distraction"  
    if re.search(r"SPEED|RACING|TOO FAST|UNSAFE SPEED", val):  
        return "Speeding"  
    if re.search(r"ALCOHOL|DRUG|CANNABIS|IMPAIR", val):  
        return "Alcohol/Drugs"  
    if re.search(r"YIELD|SIGNAL|STOP SIGN|RIGHT OF WAY", val):  
        return "Failure to Yield"  
    if re.search(r"WEATHER|FOG|RAIN|SNOW|ICE|PAVEMENT", val):  
        return "Weather/Road"  
    if re.search(r"FATIGUE|FELL ASLEEP|DROWSY", val):  
        return "Fatigue"  
    return "Other"  
  
vehicle_cols = [c for c in df.columns if "vehicle_type_code" in c]  
df["num_vehicles"] = df[vehicle_cols].notna().sum(axis=1)
```

Defining Crash Severity

1. $\text{severity_score} = \text{injuries} \times 1 + \text{fatalities} \times 10$
2. $\text{is_severe} = 1$ if total injuries ≥ 2 OR any fatality
3. is_severe is the model's prediction target
4. $\sim 42\%$ severe, $\sim 58\%$ non-severe

```
df["severity_score"] = (  
    df["number_of_persons_injured"] * 1 +  
    df["number_of_persons_killed"] * 10  
)  
  
df["is_severe"] = (  
    (df["total_injuries"] >= 2) |  
    (df["total_fatalities"] >= 1)  
) .astype(int)  
  
# drop rows missing critical info  
df = df.dropna(subset=["hour", "crash_date"])  
  
print(f"severe crashes: {df['is_severe'].sum():,} / {len(df):,}  
      ({df['is_severe'].mean():.1%})")
```

SEVERITY ANALYSIS

Severe Crashes = 41,871 / 100,000 (41.9%)

Saving to SQLite

1. Cleaned data saved as collisions_clean.csv
2. Also loaded into SQLite — table: crashes
3. Pre-aggregated summary table:
borough_hour_summary

```
conn = sqlite3.connect(db_path)

df.to_sql("crashes", conn,
         if_exists="replace", index=False)

summary_df = pd.read_sql("""
    SELECT borough, hour,
           COUNT(*) as total_crashes,
           ROUND(AVG(severity_score), 3) as avg_severity,
           SUM(is_severe) as severe_crashes
    FROM crashes
    GROUP BY borough, hour
""", conn)

summary_df.to_sql("borough_hour_summary", conn,
                 if_exists="replace", index=False)
```

SQL & Exploratory Analysis

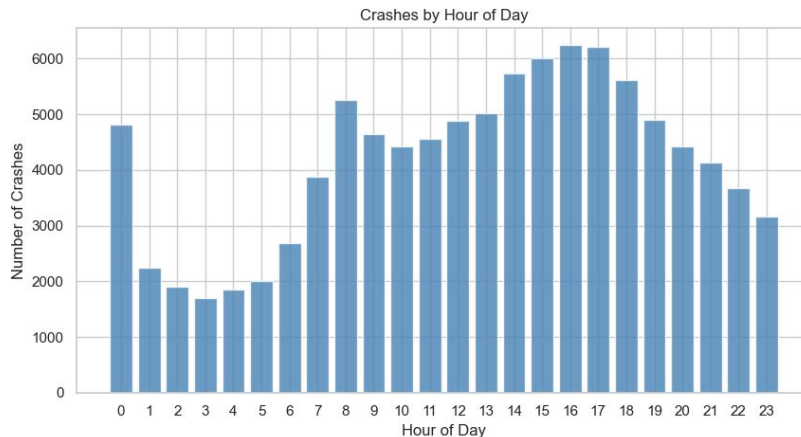
—

Plot 1: Crashes by Hour

1. `groupby('hour')` splits data into 24 groups
2. `.size()` counts rows in each group — no column needed
3. `plt.bar()` creates the bar chart
4. `plt.xticks(range(0,24))` labels every hour on x-axis
5. `plt.savefig()` saves PNG to plots folder
6. Peak: 6,241 crashes at hour 16 (4pm)
7. Rush hour appears to be the most dangerous window

```
# PLOT 1: when during the day do most crashes happen?
hourly = df.groupby("hour").size()

plt.figure(figsize=(10, 5))
plt.bar(hourly.index, hourly.values, color="steelblue", alpha=0.8)
plt.title("Crashes by Hour of Day")
plt.xlabel("Hour of Day")
plt.ylabel("Number of Crashes")
plt.xticks(range(0, 24))
plt.savefig(BASE + "\\plots\\plot1_crashes_by_hour.png")
plt.show()
print("saved plot1")
```

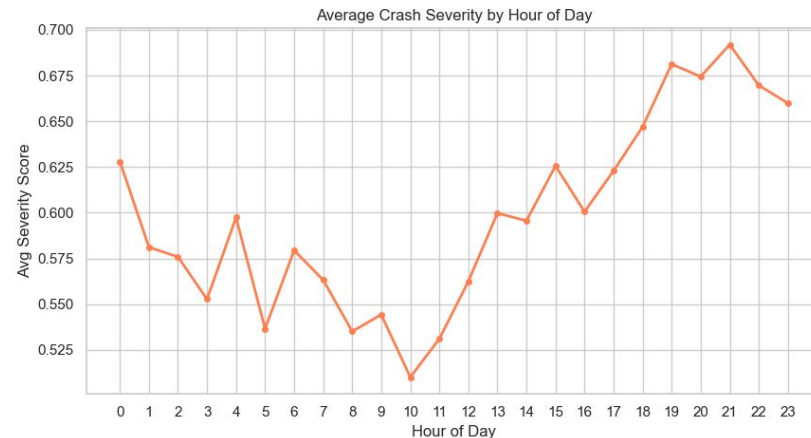


Plot 2: Severity by Hour

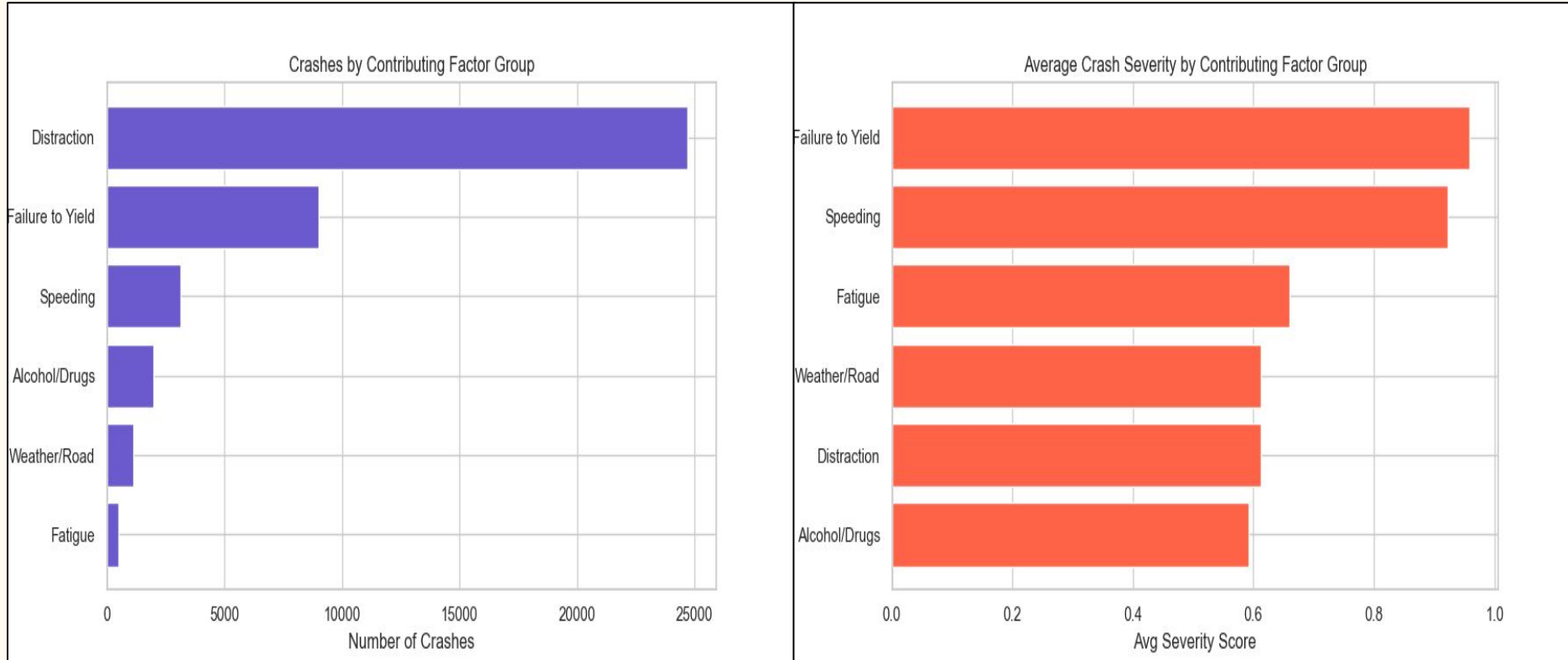
1. `groupby('hour')['severity_score']` selects one column per group
2. `.mean()` computes average — not count
3. `plt.plot()` with `marker='o'` shows trend over time
4. Line chart chosen to show progression, not just comparison
5. Lowest severity: hour 10 (0.510) — mid-morning
6. Peak severity: hour 21 / 9pm (0.692)
7. Frequency and severity are OPPOSITES here

```
# PLOT 2: does a higher crash count = more severe crashes?
hourly_sev = df.groupby("hour")["severity_score"].mean()

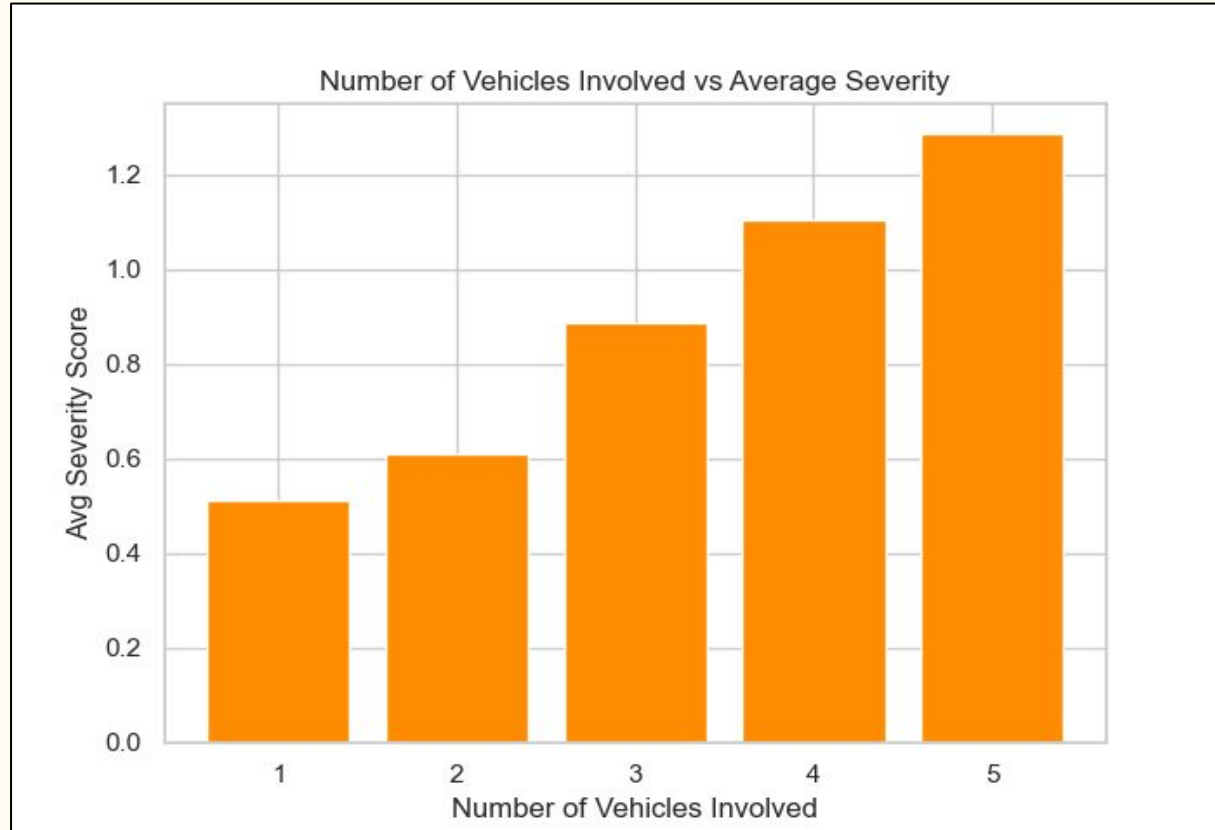
plt.figure(figsize=(10, 5))
plt.plot(hourly_sev.index, hourly_sev.values, color="coral", linewidth=2, marker="o", markersize=4)
plt.title("Average Crash Severity by Hour of Day")
plt.xlabel("Hour of Day")
plt.ylabel("Avg Severity Score")
plt.xticks(range(0, 24))
plt.savefig(BASE + "\\plots\\plot2_severity_by_hour.png")
plt.show()
print("saved plot2")
```



Plot 7 vs Plot 8: The defining difference



Plot 9: Vehicles vs Severity



SQL Helper Function: run_query

1. `conn = sqlite3.connect()` connects us to SQLite database.
2. Cursor is an object we create to help us run SQL queries
3. Now the function `run_query` executes the query, fetches all matching rows, gets the column names, and converts everything into a DataFrame for easier analysis in Python.

```
conn = sqlite3.connect(BASE + "\\data\\collisions.db")
cursor = conn.cursor()
print("Connected to database")

# make sure results folder exists
os.makedirs(BASE + "\\results", exist_ok=True)

# helper function - runs a query and returns a dataframe, same as recitation
def run_query(query, params=None):
    if params is None:
        params = ()
    cursor.execute(query, params)
    rows = cursor.fetchall()
    if len(rows) == 0:
        return pd.DataFrame()
    cols = [col[0] for col in cursor.description]
    return pd.DataFrame(rows, columns=cols)
```

Query 12: Crashes by Factor Group

```
# Q12: crashes by factor group overall
q12 = run_query("""
    SELECT
        factor_group,
        COUNT(*) as total_crashes,
        ROUND(AVG(severity_score), 3) as avg_severity,
        SUM(is_severe) as severe_crashes,
        ROUND(SUM(is_severe) * 100.0 / COUNT(*), 1) as pct_severe
    FROM crashes
    WHERE factor_group != 'Other'
    GROUP BY factor_group
    ORDER BY avg_severity DESC
""")
print("\nQ12: crashes by factor group")
print(q12)
q12.to_csv(BASE + "\\results\\queries\\q12_crashes_by_factor_group.csv", index=False)
```

factor_group	total_crashes	avg_severity	severe_crashes	pct_severe
Failure to Yield	9003	0.958	6247	69.4
Speeding	3143	0.922	1606	51.1
Fatigue	517	0.66	215	41.6
Weather/Road	1151	0.613	478	41.5
Distraction	24692	0.612	10803	43.8
Alcohol/Drugs	1978	0.592	729	36.9

Query 2: Deadliest Hours

```
# Q2: which single hour has the absolute worst average severity
# interesting because it might not be rush hour
q02 = run_query("""
SELECT
    hour,
    COUNT(*) as total_crashes,
    ROUND(AVG(severity_score), 3) as avg_severity,
    SUM(is_severe) as severe_crashes
FROM crashes
GROUP BY hour
ORDER BY avg_severity DESC
LIMIT 5
""")
print("\nQ2: deadliest hours")
print(q02)
q02.to_csv(BASE + "\\results\\queries\\q02_deadliest_hours.csv", index=False)
```

hour	total_crashes	avg_severity	severe_crashes
21	4129	0.692	1915
19	4907	0.681	2326
20	4417	0.674	2039
22	3675	0.67	1587
23	3170	0.66	1341

Query 17 Severity by Vehicle Type

```
# Q17: severity by vehicle type. Motorcycles should be way worse than sedans
q17 = run_query("""
SELECT
    vehicle_type_code1,
    COUNT(*) as total_crashes,
    ROUND(AVG(severity_score), 3) as avg_severity,
    SUM(is_severe) as severe_crashes,
    ROUND(SUM(is_severe) * 100.0 / COUNT(*), 1) as pct_severe
FROM crashes
WHERE vehicle_type_code1 NOT IN ('OTHER', 'Unknown')
GROUP BY vehicle_type_code1
HAVING COUNT(*) >= 50
ORDER BY avg_severity DESC
""")
print("\nQ17: severity by vehicle type")
print(q17)
q17.to_csv(BASE + "\\results\\queries\\q17_severity_by_vehicle_type.csv", index=False)
```

vehicle_type_code1	total_crashes	avg_severity	severe_crashes	pct_severe
STANDING S	512	1.074	118	23
MOTORCYCLE	1269	1.046	875	69
MOPED	925	1.01	781	84.4
BICYCLE	2747	0.979	2380	86.6
TANKER	72	0.764	24	33.3
TAXI	3183	0.691	1661	52.2
BUS	2072	0.626	745	36
SUV	33717	0.593	13396	39.7
SEDAN	45067	0.561	17560	39
CARRY ALL	136	0.559	41	30.1
VAN	673	0.544	243	36.1
SUBN	54	0.537	19	35.2
TRUCK	5121	0.534	1797	35.1
CONVERTIBLE	331	0.532	132	39.9
DUMP	338	0.482	93	27.5
LIMO	61	0.459	23	37.7
FLAT BED	167	0.377	41	24.6
GARBAGE OR REFUSE	289	0.356	60	20.8
PK	254	0.311	62	24.4
AMBULANCE	560	0.263	95	17

Modeling & Evaluation

—

Modeling

Evaluation of Final Results

1. ____

2. ____

3. ____

Model	Accuracy	Balanced Accuracy	AUC-ROC
Logistic Regression	0.63190	0.617598	0.664070
Gradient Boosting	0.63810	0.614864	0.673883
Random Forest	0.63755	0.606710	0.652992
AdaBoost	0.65380	0.601423	0.658456

BEST MODEL: Logistic Regression

Balanced Accuracy: 0.6176

AUC-ROC: 0.6641

Conclusion

Future Work

Potential Improvements:

Work on Branching Projects:
